

Reviewer Pack — Minimal Monoid Order and Frege Proof Depth Inequality

Entry cce8e036c2e7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 00:07:39 UTC

Minimal Monoid Order and Frege Proof Depth Inequality

Entry ID: cce8e036c2e7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 00:07:39 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Monoid Theory) **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

For every Boolean formula φ with n variables, the minimum order of a monoid M such that φ is a homomorphic image of the free monoid on n generators ($\text{min_order}(M)$) is linearly correlated with its Frege proof depth $f(\varphi)$, satisfying $\text{min_order}(M) = \Theta(f(\varphi))$.

Rationale (proposer's reasoning):

Monoids are algebraic structures that have been studied in combinatorics for their connection to symmetry and group theory. If a direct correspondence between monoid order and proof complexity can be established, it might provide new insights into the structure of proofs and potentially lead to more efficient proof systems.

Taxonomy category: algebraic_combinatorics_monoid_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 30fe47274a53bcc6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 random seeds, the correlation coefficient between $\text{min_order}(M)$ and $f(\varphi)$ is ≥ 0.95 , with no seed producing a correlation coefficient < 0.8 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Minimal Monoid Order" AND "Frege Proof Depth" - Monoid Theory AND Boolean Satisfiability - "Homomorphic Image" IN BOOLEAN MODE AND Frege proof complexity

Top relevant hits considered: - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/2312.01243v2] Boolean inverse semigroups and their type monoids - [http://arxiv.org/abs/2204.10033v4] Characterizations of classes of countable Boolean inverse monoids - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras - [http://arxiv.org/abs/0903.3848v1] Join-irreducible Boolean functions - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [s2:10.1007/s00224-016-9712-2] The Half-Levels of the FO2 Alternation Hierarchy - [s2:10.1109/LICS.2004.52] VTC: A Second-Order Theory for TC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_boolean_formula(n):
    if n == 0:
        return "True"
    elif n == 1:
        return "False"

    op = random.choice(["&", "|"])
    left = generate_boolean_formula(random.randint(0, n//2))
    right = generate_boolean_formula(n - len(left.split("&")) - len(right.split("|")))
    return f"({left} {op} {right})"

def min_order(n):
    if n == 1:
        return 1
    else:
        return 2 * min_order(n // 2) + 1

def frege_proof_depth(formula):
    if formula in ["True", "False"]:
        return 0
    elif "&" in formula:
        left, right = formula.split("&")
        return max(frege_proof_depth(left), frege_proof_depth(right)) + 1
    else:
        return 0
```

```

        left, right = formula.split("|")
        return max(frege_proof_depth(left), frege_proof_depth(right)) + 1

def run_trial(seed: int) -> dict:
    random.seed(seed)

    metric_name = "correlation_coefficient"
    instances_tested = 0
    n_max = 0
    min_order_values = []
    frege_depth_values = []

    for n in [5, 10, 15, 20, 30, 40]:
        formula = generate_boolean_formula(n)
        if formula:
            instances_tested += 1
            n_max = max(n_max, n)
            min_order_value = min_order(n)
            frege_depth_value = frege_proof_depth(formula)
            min_order_values.append(min_order_value)
            frege_depth_values.append(frege_depth_value)

    if not min_order_values or not frege_depth_values:
        return {
            "metric_name": metric_name,
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    mean_min_order = sum(min_order_values) / len(min_order_values)
    mean_frege_depth = sum(frege_depth_values) / len(frege_depth_values)
    correlation_coefficient = (len(min_order_values) * sum(a * b for a, b in zip(min_order_values,
                                                                              sum(min_order_values) * sum(frege_depth_values)) / \
                                                                              math.sqrt((len(min_order_values) * sum(x**2 for x in min_order_valu
                                                                              (len(frege_depth_values) * sum(y**2 for y in frege_depth

    return {
        "metric_name": metric_name,
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": correlation_coefficient >= 0.95 and all(corr >= 0.8 for corr in [correlation_coefficient]),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}, \"instances_tested\": {result['instances_tested']}, \"n_max\": {result['n_max']}, \"conjecture_holds\": {result['conjecture_holds']}, \"counterexample\": \"{result['counterexample']}\"}}")

```

```

results.append(result)

mean_metric_value = sum(res["metric_value"] for res in results if res["metric_value"] is not None)
std_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value)**2 for res in results) / len(results))
support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not res["conjecture_holds"] for res in results) and any(res["metric_value"] < 0.8 for res in results):
    first_failing_seed = next((seed for seed, res in zip(seeds, results) if not res["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7932f364.py", line 96, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7932f364.py", line 55, in run_trial
    formula = generate_boolean_formula(n)
               ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7932f364.py", line 25, in generate_boolean_formula
    left = generate_boolean_formula(random.randint(0, n//2))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7932f364.py", line 26, in generate_boolean_formula
    right = generate_boolean_formula(n - len(left.split("&")) - len(right.split("|")))
           ^^^^^
UnboundLocalError: cannot access local variable 'right' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the correlation coefficient between $\min_order(M)$ and $f(\phi)$. As a result, we cannot confirm or refute the conjecture based on the provided evidence. | next: Investigate the cause of the crash in the test code to ensure it can run successfully. Once the issue is resolved, re-run the test with the specified conditions to evaluate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17138
2	propose	ollama_remote	glm4:latest	0	0	12734
3	preregistration	ollama_remote	glm4:latest	0	0	9879
4	novelty	ollama_remote	glm4:latest	0	0	8423
5	novelty	ollama_remote	glm4:latest	0	0	22539
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	49573
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11094
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12780
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13051
10	judge	ollama_remote	glm4:latest	0	0	22209

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 179418 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/cce8e036c2e7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cce8e036c2e7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cce8e036c2e7.tar.gz` (if generated)